Yaw-Rate Latency, Measured Three Ways

Commanded vs IMU vs diff_drive_controller — iteration 3 bags, runs 5 to 9

The answer

The IMU and the **raw wheel encoders** agree on when the robot started turning to within **0.7 ± 3.7 ms**. Neither reporting path is slow. Your instinct that the diff-drive figure travels a slower road is right, but the slow part is not the wiring.

The **published** `/diffbot_base_controller/odom` yaw rate trails the raw encoders by **154.1 ± 2.8 ms**, and that is entirely `velocity_rolling_window_size`, which is not set in `diffbot_controllers.yaml` and so defaults to **10**. Ten samples at the 30 Hz `update_rate` is (10−1)/2 / 30 = **150 ms** of group delay. Predicted 150, measured 154.

So the ~**325 ms** from command to motion is **real drivetrain dead time**, not a measurement artefact — two sensors on completely different paths agree on it. The extra 155 ms is a moving average you can delete with one line.

1. How this was measured

Everything is on **header stamps**, not bag receive times. That matters: the commanded TwistStamped carries a stamp 32–69 ms older than the moment it is received, so using receive times quietly hides the first chunk of the budget. `/imu/data` and `/diffbot_base_controller/odom` are both stamped within 0.2 ms of publication, so their stamps are directly comparable.

Four channels, all against the same commanded yaw rate:

channel	topic	what it is
IMU	/imu/data	the body's real rotation, over USB
raw wheels	/dynamic_joint_states	the wheel velocity state interfaces, straight off the serial link, *before* diff_drive_controller turns them into odometry
published odometry	/diffbot_base_controller/odom	the yaw rate diff_drive_controller derives and publishes
EKF output	/odometry/filtered/local	what `controller_server` actually reads back

Three estimators, because a single correlation peak is easy to fool when two signals have different shapes: the **cross-correlation peak** with a parabolic refinement; a **first-order-plus-dead-time fit**, $y = K \cdot \text{lowpass}_\tau(u(t - L))$, which splits the response into dead time L where nothing happens and a lag τ where it happens slowly; and the **group delay** from the slope of the cross-spectral phase. Signals are resampled onto a 5 ms grid and restricted to the stretch where the controller was commanding.

Caveats worth knowing before you use these numbers

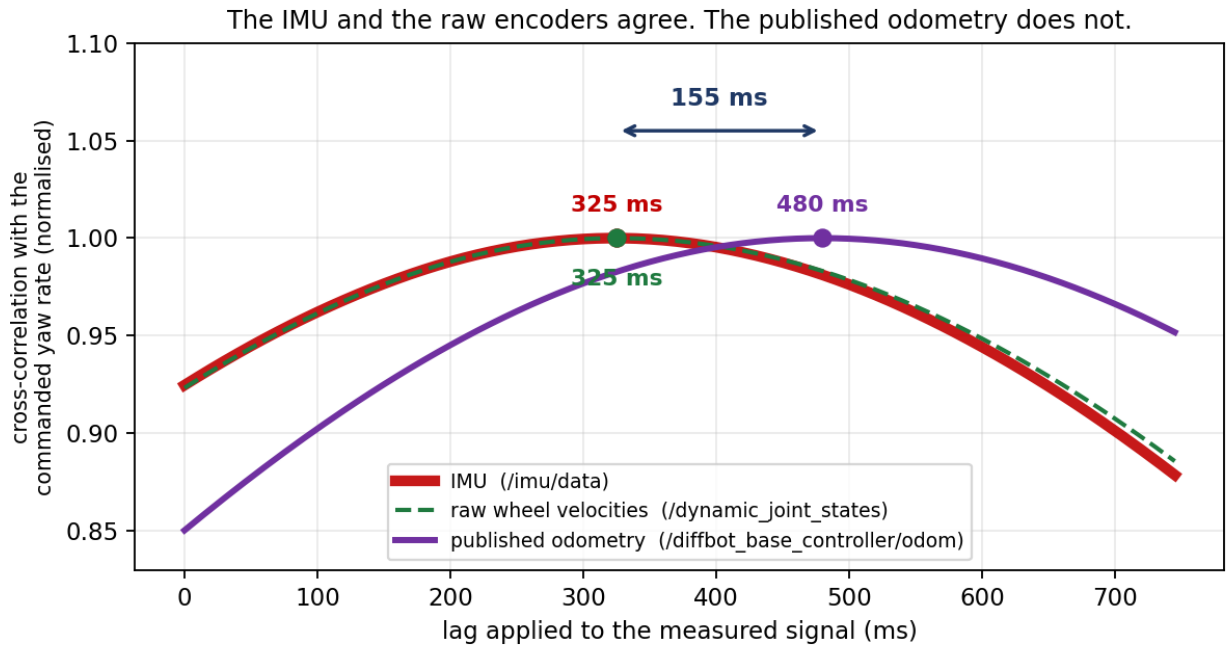
The IMU publishes at **19.5 Hz** (51 ms period) and the odometry at 30 Hz. A slower sensor biases its apparent lag *later*, so this works against the headline finding, not for it — the IMU still comes out 155 ms earlier than the published odometry.

L and τ trade off against each other in the fit; individually they wander (τ from 30 to 210 ms across runs) while the peak and the group delay stay tight. Trust the totals, treat the L / τ split as indicative.

The **linear** axis could not be measured usefully from these runs: the peak lands at 50–72 ms but with a ±28–49 ms spread, because vx varies too slowly to localise a delay. Structurally it is subject to the same rolling mean, so expect it to show up under hard acceleration.

These bags predate both of your recent changes — `wheel_separation` 1.016 → 0.856 and the `rtlatency` firmware instrument. The gains below reflect 1.016; the *timings* are unaffected by either.

2. What came out

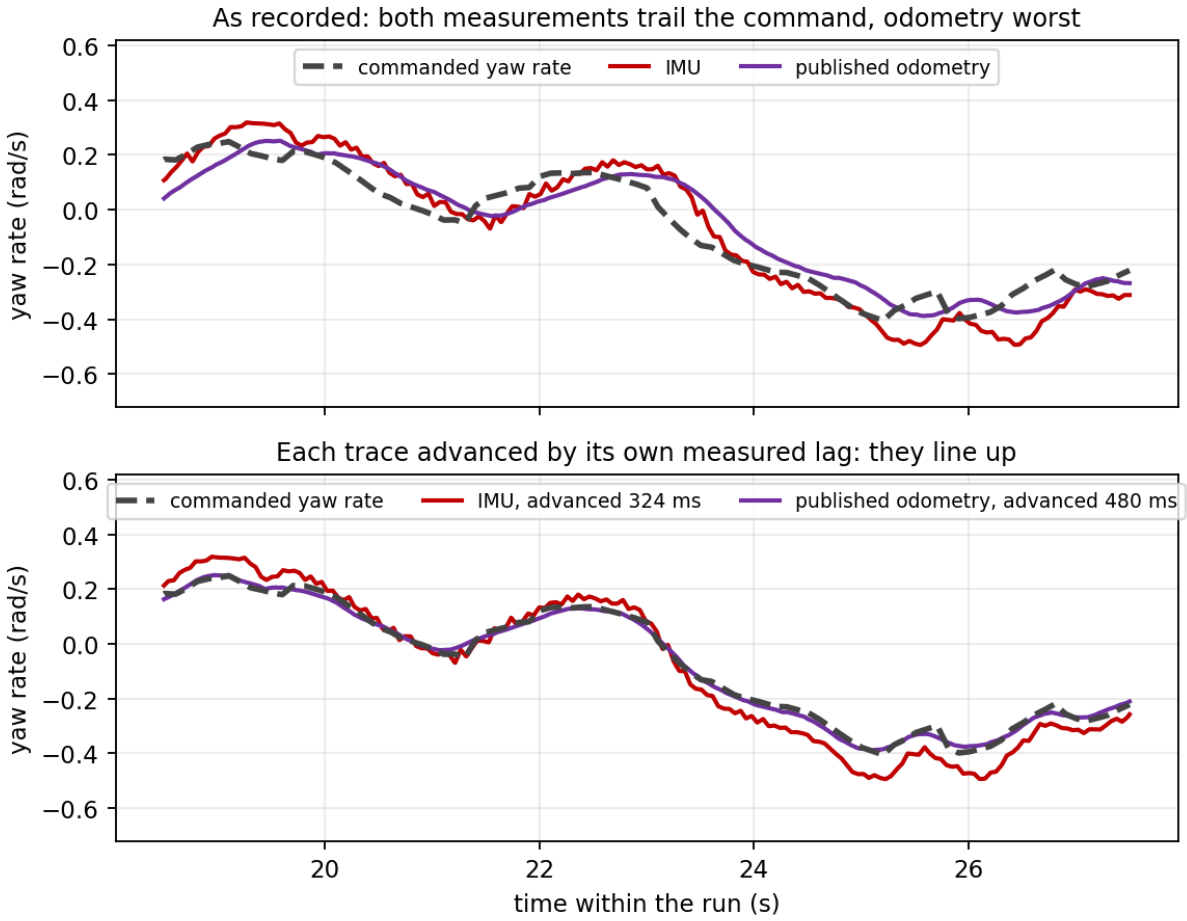


Run 7. The IMU (thick red) and the raw wheel velocities (dashed green) lie on top of each other and peak together at 325 ms. The published odometry peaks 155 ms later.

channel	peak lag	group delay	dead time L	lag τ	gain
IMU	327 ± 31 ms	350 ± 19 ms	254 ± 31 ms	86 ± 42 ms	1.24
raw wheels	328 ± 28 ms	347 ± 9 ms	216 ± 31 ms	130 ± 51 ms	—
EKF output	329 ± 30 ms	352 ± 19 ms	257 ± 31 ms	88 ± 40 ms	1.24
published odometry	482 ± 26 ms	496 ± 8 ms	361 ± 54 ms	138 ± 69 ms	0.99

Mean ± standard deviation over runs 5–9. The raw-wheel channel is a difference of wheel velocities in rad/s, so its gain is not comparable and is omitted; only its timing is used. The FOPDT fit reaches r^2 0.93 on average.

difference	mean ± sd over the five runs
raw wheels – IMU	**+0.7 ± 3.7 ms** (the two sensors agree)
EKF output – IMU	+2.6 ms (the EKF's yaw rate *is* the IMU)
published odometry – raw wheels	**+154.1 ± 2.8 ms**
published odometry – IMU	+154.8 ± 6.4 ms



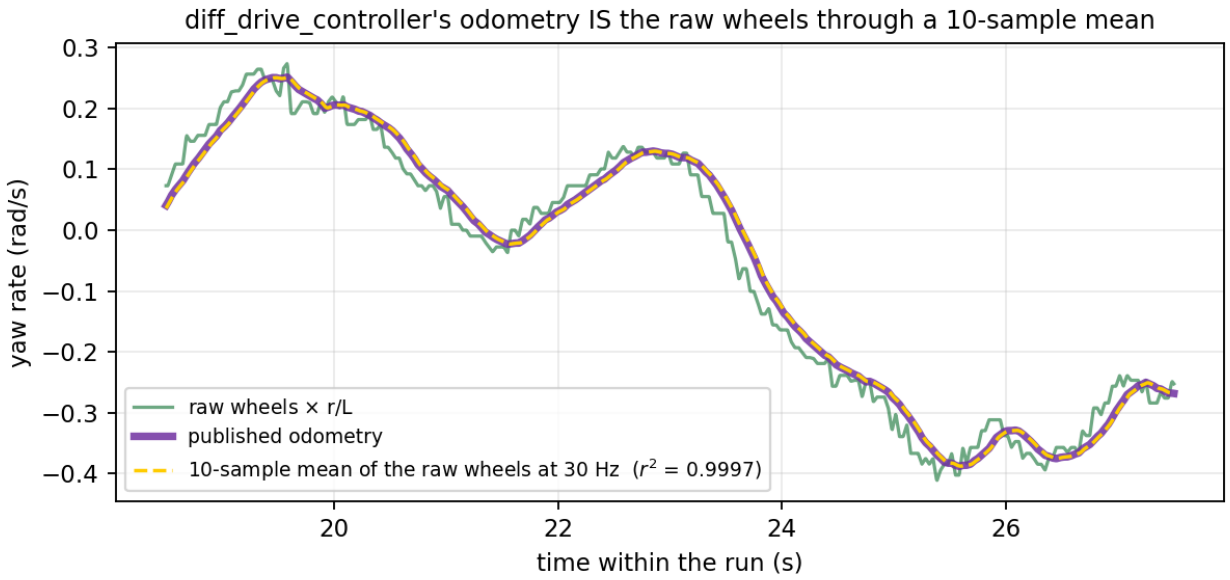
Run 7, the busiest nine seconds. Top: as recorded. Bottom: each trace pulled forward by its own measured lag, which collapses them onto the command — the lags are real and they are close to pure delay.

3. Where the 155 ms comes from

`diff_drive_controller` does not publish the instantaneous wheel velocity. Its `Odometry` class runs each axis through a `RollingMeanAccumulator` of `velocity_rolling_window_size` samples. Your `diffbot_controllers.yaml` does not set it, and the default in `diff_drive_controller_parameters.hpp` is **10**. The accumulator is fed once per `controller_manager` cycle, and you have `update_rate: 30`.

```
group delay of an N-sample moving average = (N − 1) / 2 / rate
                                          = (10 − 1) / 2 / 30 Hz = 150 ms
```

That is the prediction. To check it rather than assume it, the raw wheel signal was passed through exactly that filter and compared with the published odometry:



The yellow dashed line is a 10-sample mean of the raw wheel velocities at 30 Hz, scaled by r/L. It sits on the published odometry with r² = 0.9997 and no residual lag.

Across runs 5–9 the reconstruction leaves a residual lag of –**15 to 0 ms** (one 5 ms grid step or less) and a correlation of **0.9998 to 1.0000**. The fitted scale comes out at **0.1624**, against r/L = 0.165 / 1.016 = 0.1624 — which also confirms the channels are what they are labelled, and that these bags were recorded on the old `wheel_separation`. The published odometry yaw rate is, to within measurement resolution, the raw wheels through that moving average. Nothing else is going on.

4. The budget, and what it means

segment	cost	measured how
MPPI stamps the command → it reaches the base	32–69 ms	header stamp vs bag receive time, directly
→ the wheels and the body actually respond	~255 ms dead + ~90–130 ms first-order (**325 ms** to the correlation peak)	IMU and raw encoders, independently, agreeing
→ it appears in the published odometry	**+155 ms**	the rolling mean, reproduced exactly

What is already fine

The local EKF takes **vyaw from the IMU only** (`imu0_config` index 11 true, `odom0_config` index 11 false), and the measured EKF output lag (329 ms) matches the IMU (327 ms). Since `controller_server` reads `odometry/filtered/local`, **MPPI's yaw-rate feedback is not affected by the rolling mean.** Good news, and worth knowing before anyone spends time on it.

What is not

Linear velocity is. `odom0_config` fuses vx and vy from `/diffbot_base_controller/odom` — the smoothed one — so the vx MPPI starts its rollouts from is filtered through the same 150 ms. These runs could not measure it (see the caveats), but structurally it is there, and it will matter most exactly when it hurts: hard acceleration and emergency stops.

Anything reading the odometry yaw rate directly gets a 155 ms-stale value: RViz, the lap manager, the speed governor, any logging you judge behaviour from. Nothing in the control loop does today, but it is a trap worth labelling.

5. Suggestions

All of these are outside `Course2027.yaml` and `recovery_nav_bt.xml`, so nothing here has been changed.

	where	what
1	`diffbot_controllers.yaml`, `diffbot_base_controller`	Set `velocity_rolling_window_size` explicitly. At `update_rate: 30`, a window of 2 gives 17 ms of group delay and 4 gives 50 ms, against 150 ms today. Less smoothing means a noisier vx into the EKF, so **4 is the safer first step** rather than 2 — it recovers 100 of the 150 ms and keeps some filtering. Watch the EKF's vx for chatter afterwards.
2	the drivetrain	The ~255 ms of dead time is the real prize and it is genuine — two independent sensors agree. Your new `rtlatency` covers the board A → B → A round trip, which is one leg of it; the rest is the host serial write, the 30 Hz `controller_manager` tick (up to 33 ms on its own), board B → motor controllers, and the motors themselves. Record `/dynamic_joint_states` next run and `rtlatency` comes along with it for free, since it is now a GPIO state interface.
3	`diffbot_controllers.yaml` vs `Course2027.yaml`	`angular.z.max_acceleration` is **1.5** rad/s² in the base controller while MPPI plans with `az_max: 2.0`. The base silently rate-limits what MPPI asks for, which shows up as extra τ and as the controller mispredicting its own trajectories. Make the two agree — either number is defensible, disagreeing is not.
4	MPPI, once the above is known	If the ~300 ms cannot be reduced, it can at least be told to the controller. MPPI assumes its command takes effect immediately; at 0.5 m/s, 300 ms is 15 cm of travel committed before anything happens. This is the mechanism behind the weaving in the iteration 3 report.

Tooling: `claude_tools/latency_breakdown.py` produces every number in this report, and `claude_tools/make_figures_latency.py` the figures. Re-run them on the next set of bags to see whether the window change lands.